Kartavya AI-” Pass on what matters, with proof ”

An evidence-linked, permission-aware knowledge handover platform that turns scattered project activity into a trustworthy continuation plan when a team member leaves.

CHALLENGE	Prevent Critical Knowledge From Leaving With People and task progression easier
TEAM NAME	Team Raahat

PROBLEM STATEMENT-

When team members leave or change roles, the context behind their work leaves too. It sits scattered across documents, emails, chats and repositories, and manual handovers are slow, inconsistent and incomplete.

Impact

- Successors struggle to understand existing work
- Projects stall and past mistakes repeat
- Knowledge is lost when the leaver is unavailable
- Generic summarisers can’t cite sources, flag conflicts or protect confidential data

The slice we solve
When someone leaves a small software or project team, the incoming person cannot quickly learn what was decided, who owns what, what is unfinished and what is risky, without missing context or seeing confidential material.

- **Decision loss:** reasoning is buried in threads and pull requests
- **Ownership ambiguity:** boards, code and chat disagree on who owns what
- **Stalled tasks:** half-finished work has no clear state, next step or owner
- **Silent gaps:** the leaver knows things no source records
- **Unsafe sharing:** naive summaries leak confidential discussions

TARGET USERS-

Who experiences the problem
• Teams with high turnover: student clubs, hackathon organiser groups, volunteer communities and NGOs (annual leadership change).
• Small software and research teams where one person holds most of the context (low bus factor).
• Open-source projects when a maintainer steps back.

• Managers who must find a replacement under time pressure.

User	Needs	How they use Kartavya
Departing member	Hand over without writing a long document from scratch.	Reviews AI-reconstructed state, answers targeted gap questions, corrects errors.
Approver / manager	Confidence that the handover is accurate and safe to share.	Approves, rejects or edits claims; sets who sees what; tracks version history.
Incoming member (developer / coordinator)	Fast, trustworthy context and a clear first-30-days path.	Reads a role-appropriate handover, clicks a claim to open the source, asks grounded questions, acknowledges receipt.
Admin (secondary)	Keep access control and audit trails correct.	Configures source permissions and review rules.

OUR SOLUTION-

Kartavya pulls in authorised project sources and builds a clear picture of where the project stands: decisions, owners, open questions, risks, dependencies and next steps. Every claim comes with evidence and a confidence label. A human reviewer corrects and approves it, and the incoming person then gets a view tailored to their role, with sensitive details redacted.

What makes it more than a summariser

- **Decision timeline:** shows what was decided, the alternatives, who decided, why, and whether it was later contradicted or left open.
- **Evidence-first claims:** every claim is labelled Verified (direct source), Inferred (reasoned from sources), Conflicting (sources disagree) or Missing evidence. Nothing appears unlabelled.
- **Questions before they leave:** each gap or conflict becomes a short, specific question for the departing person, so missing knowledge is recovered in a 15-minute chat instead of being lost.
- **Permission-aware views:** every source and fact carries an access label. The newcomer sees placeholders like “2 related items hidden. Request access.” and never the content itself.
- **Grounded Q&A:** the newcomer can ask follow-up questions. Answers come only from approved, permitted evidence, and the system says “no evidence found” when that’s the case.

COMPLETE PRODUCT WORKFLOW-

Stage	What happens	Key Output	Trust gate
1. Connect	The admin opens a case (project, departing member, successor and role) and adds authorised GitHub activity, notes, CSV boards and chat-export files. Each source receives a consent record and access label.	A handover case and labelled source inventory.	Ingest only sources the team is authorised to use. No live employee monitoring.
2. Reconstruct	The system parses, chunks and indexes content, then extracts decisions and rationale, owners, task states, blockers, dependencies, risks, open questions and next actions as structured records with a source ID and exact quote span.	A draft project-state model and decision/task timeline.	Reject any extracted item that has no valid source span.
3. Verify	A verification pass checks whether each cited span supports its claim and assigns Verified, Inferred, Conflicting or Missing evidence status. Related records are grouped for review.	Evidence-linked claims plus a queue of gaps and possible conflicts.	Never present unsupported claims as facts. Automated cross-source conflict detection is a Round 3 improvement.
4. Review	The departing member and approver accept, edit, reject or comment on each claim and add attributed clarifications. Every change is saved as a new version.	A corrected, versioned handover ready for approval.	A human approver must approve the handover before it is shared.
5. Handover	The system prepares a role-appropriate view from approved information, showing task owner, last known state, blocker and next action. The successor opens citations and acknowledges receipt.	A successor-ready handover, task continuation list and receipt record.	Check access before delivery and log the handover. Full per-claim redaction and grounded Q&A are Round 3 improvements; the 30-day plan is a stretch item.

MAIN FEATURES WE INTEND TO BUILD-

#	Feature	What it delivers	Round / priority
1	Multi-source ingestion	Import authorised GitHub issues, pull requests and commits; Markdown/PDF notes; CSV task boards; and chat-export JSON/text.	Round 2
2	Project-state extraction	Extract decisions and rationale, owners, task states, blockers, dependencies, risks, open questions and next actions into structured records.	Round 2
3	Evidence-linked citations	Attach a source ID and exact quote span to every displayed claim; clicking a claim opens the supporting passage.	Round 2
4	Claim evidence labels	Show Verified, Inferred, Conflicting or Missing evidence on each claim; keep inference visibly distinct from verified fact.	Round 2
5	Decision timeline	Show decisions in chronological order, including recorded rationale, alternatives, known reversals and unresolved threads.	Round 2
6	Review and approval board	Let the departing member and approver accept, edit, reject or comment on claims; retain version history and require approval before sharing.	Round 2
7	Role-based handover view	Show approved, role-appropriate facts to developers, managers and coordinators; carry task owner, last state, blocker and next action; capture acknowledgement.	Round 2
8	Conflict and uncertainty detection	Compare related claims across sources, explain disagreements and surface unresolved gaps instead of silently merging conflicting information.	Round 3 Committed
9	Permission-aware redaction	Filter source- and claim-restricted facts before any LLM prompt or successor view; show redacted placeholders, support access requests and keep an audit log.	Round 3 Committed
10	Grounded Q&A and gap questions	Answer only from approved, permitted evidence with citations; return “no evidence found” when unsupported and generate targeted questions for the departing member.	Round 3 Committed
11	30-day continuation plan	Prioritise first-month tasks, key people to meet and checkpoints, with acknowledgement tracking.	Round 3 Stretch

TECHNOLOGY STACK-

Layer	Choice
Frontend	React, Next.js, TypeScript, Tailwind CSS
Backend / API	Python FastAPI and Pydantic
Database	PostgreSQL and pgvector
AI / LLM	Hosted LLM API with structured JSON output; embedding model for retrieval
Ingestion	GitHub REST API and Markdown/PDF/CSV/JSON parsers
Auth and access control	JWT, role-based access, source-level and claim-level access labels
Background jobs	RQ or Celery with Redis
Deployment	Docker Compose; frontend on Vercel; backend/database on a managed host
Testing / tooling	Pytest, Playwright, GitHub Actions

Why This Technology Stack-

- FastAPI + Pydantic: Reliable APIs and data validation.
- PostgreSQL + pgvector: Secure storage and smart semantic search.
- React + Next.js: Interactive dashboards and evidence tracking.
- API Permissions: Protect sensitive data and control access.
- Docker Compose: Simple setup, testing, and deployment.

Built for speed, security, scalability, and reliability.

MAJOR TECHNICAL CHALLENGE AND OUR APPROACH-

Challenge	Risk	Solution
AI hallucinations	The AI generates false statements or claims tasks are completed when they aren't.	**Evidence Verification** — Every claim must cite a Claim ID,

		Source ID, and exact text span. Unbacked claims are marked *Unsupported* and hidden.
Conflicting Information	Different sources may state contradictory facts (e.g., meeting notes assigning a task to one person while a GitHub issue assigns it to another). AI should not	**Conflict detection.** The system highlights both versions, labels the claim as "Conflicting," and flags it for human review and resolution
Confidential Data Leaks	Hiding restricted info *after* generating text risks accidental data exposure to junior team members.	**Permission-Aware Retrieval** — Filters sources and facts at the backend API/database level *before* generating role-specific views.
Over-Reliance on AI Output:-	Blindly trusting AI-generated handovers leads to errors and poor decision-making.	**Human-in-the-Loop** — Requires mandatory review and approval by the departing lead or manager before the handover is published.
Missing Context	Incomplete docs leave gaps, causing the AI to guess decisions or status.	**Targeted Questioning** — Identifies knowledge gaps and automatically prompts the departing member to answer critical missing details.

RESPONSIBILITIES OF EVERY TEAM MEMBER-

MEMBER	PRIMARY ROLE	RESPONSIBILITIES
Sharthak Ravi	Frontend, UX and research lead	Timeline, evidence panel, review board, role-based handover view, onboarding flow, accessibility, user interviews, testing, documentation and demo video.

Ranvijay Narayan Singh	Backend, data and security engineer	FastAPI services, database schema, ingestion connectors, version history, audit log, access-control model, redaction engine, deployment.
D.S.S.Srikar	Team lead and AI / extraction engineer	Project coordination, extraction and verification pipeline, prompt and schema design, synthetic dataset, evaluation set, conflict detection.

IMPACT-

- **Faster onboarding**: the successor starts with reviewed context instead of searching from scratch.
- **Fewer repeated mistakes**: decision history preserves why choices were made.
- Lower knowledge risk: single-owner knowledge, blockers, and unfinished work become visible.
- **Safer collaboration**: role-based access reduces accidental disclosure.
- **Success metrics**: valid citations for all displayed claims; conflicts correctly flagged; zero restricted items in unauthorised views; a new user completes the workflow without guidance.

Future Scope:-

- Live connectors for Slack, Microsoft Teams, Jira, Notion, Google Drive and email with organisation-level OAuth and permission sync.
- Proactive knowledge-risk scoring (bus-factor dashboard) that flags areas known by only one person before anyone resigns.
- Continuous handover: keep the project-state model updated so a handover is always one approval away.
- Self-hosted or on-premise deployment with local models for privacy-sensitive organisations.
- Templates for specific settings such as student clubs, NGOs and open-source maintainer transitions.
- Multilingual sources and handover output.
- Analytics on recurring problems across handovers to improve team processes.

Review tracker

Full name	Email	Institution/Year	Phone	GitHub / LinkedIn
DSS Srikar	dsssrikar…	SRM Institute of Science and Technology	9866111929	www.linkedin.com/in/dsssrikar
Ranvijay Si…	singhrvn0…	SRM Institute of Science and Technology	9205826211	https://github.com/LeotheLion-07
Sharthak Ravi	mr.agent.…	SRM Institute of Science and Technology	9263567737	https://github.com/sharthak-coder